

Генетические и меметические алгоритмы для решения задачи TSP

Пучков Пётр

18 мая 2026 г.

Аннотация

Настоящая работа посвящена применению генетических и меметических алгоритмов к NP-трудной задаче коммивояжёра (TSP). В ходе работы будут изучены различные виды мутаций, скрещивания и отбора. Кроме того будут рассмотрены гибридные методы, сочетающие в себе генетические алгоритмы и локальную оптимизацию. Изученные алгоритмы будут реализованы и протестированы. На основе полученных во время тестирования данных, будет проведён сравнительный анализ эффективности выбранных эвристик.

Содержание

1	Введение	3
2	Техническая часть	4
2.1	Формальная постановка задачи	4
2.2	Принципы работы генетических алгоритмов	4
2.3	Механизмы контроля популяции	4
2.4	Оператор селекции (отбора)	5
2.4.1	Пропорциональная селекция	5
2.4.2	Турнирная селекция	5
2.5	Оператор скрещивания (кроссовера)	6
2.5.1	Стандартный однородный кроссовер, адаптированный для перестановок	7
2.5.2	Двухточечный кроссовер (Order Crossover, OX) . . .	7
2.5.3	Усредняющий кроссовер (Particle-Like Crossover) . .	8
2.6	Оператор мутации	8
2.6.1	Обмен двух городов (Swap)	9
2.6.2	Инверсия участка маршрута (Inversion / 2-opt) . . .	9
2.6.3	Перемешивание подпоследовательности (Scramble) .	9
2.6.4	Параметры мутации	9
2.7	Меметические алгоритмы	9
3	Реализация алгоритма	10
3.1	Архитектура	10
3.2	Конфигурирование тестовых запусков через JSON	11
3.3	Сбор метрик и визуализация	12
4	Экспериментальное исследование	13
4.1	Вероятность локального поиска	13
4.2	Сравнение операторов мутации	14
4.3	Островная модель против простой популяции	16
4.4	Влияние размера популяции	17
4.5	Классический цикл против монотонного	19
4.6	Сравнение операторов кроссовера	19
4.7	Операторы селекции	21
4.8	Инициализация популяции	21
4.9	Промежуточные выводы	22
4.10	Масштабирование на 574 города	23
4.11	Общие выводы	26
5	Заключение	27

1 Введение

Задача коммивояжёра (Traveling Salesman Problem, **TSP**) - это одна из классических задач комбинаторной оптимизации, цель которой — найти кратчайший или самый дешёвый маршрут, проходящий через все заданные пункты (города) ровно по одному разу и возвращающий в исходный пункт. Задача **TSP** входит в число **NP**-трудных задач.

Для решения подобных задач не существует алгоритмов, работающих за полиномиальное время. Поэтому для их решения применяются метаэвристические подходы - они позволяют за сравнительно небольшое время получить результат, сравнимый с наилучшим решением. В данной работе будут рассмотрены генетические алгоритмы (**GA**), имитирующие процессы биологической эволюции. В отличие от классических алгоритмов оптимизации, в **GA** оптимизируется не одно решение, а набор решений, который называется популяцией. В рамках **GA** решения (маршруты) подвергаются операциям скрещивания, мутации и отбора, что позволяет эффективно исследовать пространство состояний.

К сожалению, классические реализации **GA** часто сталкиваются с проблемой «преждевременной сходимости» (отсутствие разнообразия) и низкой точности при поиске в локальных областях. Для решения этой проблемы используются меметические алгоритмы **MA**, которые дополняют генетический поиск процедурами локального улучшения.

В работе рассмотрены наиболее популярные алгоритмы скрещивания, мутации, отбора. Данные алгоритмы реализованы на языке Kotlin. Также проведён сравнительный анализ их скорости и эффективности при использовании в различных сочетаниях и разными параметрами.

2 Техническая часть

2.1 Формальная постановка задачи

Дан граф $G(V, E)$ вместе с функцией $\omega : E \rightarrow \mathbb{R}_+$, которая задаёт веса на рёбрах. Необходимо найти наименьший по длине гамильтонов цикл.

Мы будем рассматривать менее общую задачу, которая тоже является **NP**-трудной. А именно, в нашей задаче вершинами графа будут точки на плоскости, а длинами рёбер - евклидовы расстояния между ними.

2.2 Принципы работы генетических алгоритмов

В TSP алгоритм оптимизирует популяцию - набор из нескольких гамильтоновых циклов, выполняя несколько итераций оптимизации. Стоит отметить, что в таком случае не одно решение движется по пространству решений, а набор решений, который движется по пространству популяций. На каждой итерации согласно оператору селекции выбираются решения, которые будут подвержены скрещиванию, а также выбираются решения, которые будут удалены из популяции. При таком подходе за небольшое количество итераций может произойти вырождение популяции - все решения станут практически идентичными. Чтобы избежать этой проблемы, после скрещивания к каждому решению с некоторой вероятностью применяется оператор мутации. Такие итерации повторяются до тех пор, пока не будет достигнуто условие остановки.

2.3 Механизмы контроля популяции

С точки зрения динамики, в GA эволюционирует не отдельный индивид, а распределение вероятностей перехода из одной популяции в другую. Можно выделить две основные схемы управления поколениями.

- Классический GA. В начале каждой итерации (поколения) формируется промежуточная популяция одинакового с текущей размера путём применения оператора селекции. Затем к отобранным особям применяются рекомбинация и мутация, в результате чего создаётся полностью новое поколение. При такой схеме часть лучших решений может теряться, поэтому часто используется *элитизм*: несколько лучших особей гарантированно переносятся в следующее поколение без изменений.
- Стационарный (монотонный) GA. На каждом шаге вместо создания целого поколения генерируется один (или небольшое число)

потомков. Новый индивид оценивается и помещается в популяцию, при этом обычно удаляется наихудшая особь текущей популяции. Это обеспечивает монотонное неухудшение рекорда, увеличивает селективное давление, но быстрее снижает разнообразие. Реализация эффективна при хранении популяции в виде кучи, что позволяет быстро находить и заменять худшего индивида.

2.4 Оператор селекции (отбора)

Целью оператора селекции является выбор из популяции решений, которые будут далее подвержены скрещиванию. Существует несколько основных операторов отбора.

2.4.1 Пропорциональная селекция

Данный оператор селекции предложен одним из основоположников ГА, Дж. Холландом в рамках классического ГА. Обозначим приспособленность (значение оптимизируемой функции) i -ого решения за f_i и среднюю приспособленность за $\bar{f}$. При пропорциональной селекции i -ое решение выбирается для селекции с вероятностью, пропорциональной приспособленности, т.е. с вероятностью $\frac{f_i}{\bar{f}}$. Стоит отметить, что здесь подразумевается, что приспособленность максимизируется, что неверно для суммарной длины цикла в задаче TSP. Поэтому в качестве f_i предлагается рассматривать величину, обратную длине цикла.

У данного метода есть существенные недостатки:

- При наличии решения с высокой приспособленностью высока вероятность скорого вырождения популяции
- Наоборот, на поздних этапах работы алгоритма, когда популяция близка к искомому результату, особи имеют почти одинаковую приспособленность, поэтому сходимость значительно замедлится

2.4.2 Турнирная селекция

Недостатки пропорциональной селекции демонстрируют, что при селекции стоит учитывать не точное значение приспособленности, а только факт того, какое решение более приспособлено. Данный подход применяется в турнирной селекции - самом распространённом операторе отбора. Простейший вариант турнирной селекции — бинарный турнир: случайно выбираются две особи из популяции, сравниваются их значения целевой функции, и в промежуточную популяцию помещается лучшая из двух.

Такой турнир повторяется, полностью заполняя промежуточную популяцию.

Для устранения погрешностей случайного семплирования применяют несмещённый турнир: генерируется случайная перестановка индексов популяции π , после чего последовательно проводятся состязания между особями с индексами π_i и π_{i+1} . Такой приём гарантирует, что каждый индивид примет участие ровно в двух турнирах, снижая шум отбора.

Для контроля над давлением отбора вводится параметр $0.5 \leq S \leq 1$, который задаёт вероятность выбора лучшей особи. При проведении турнира между двумя особями с вероятностью S в итоговую популяцию будет взята более приспособленная особь, а иначе - менее приспособленная. При этом реальное селективное давление составляет $2S$. Значение $S = 0.5$ соответствует полностью случайному выбору (давление 1), $S = 1$ — максимальному давлению 2.

2.5 Оператор скрещивания (кроссовера)



Рис. 1: Брачный период божьих коровок. Иллюстрация взята из [2], где она используется для иллюстрации баланса между сохранением сходства родителей и поддержанием генетического разнообразия в популяции.

Оператор скрещивания моделирует рекомбинацию генов. Он совмещает два родительских решения с целью получить одного или нескольких потомков, наследующих полезные признаки обоих родителей. Для задачи коммивояжёра решение представляет собой перестановку городов (гамильтонов цикл), поэтому стандартные операторы, работающие с бинарными строками, непосредственно неприменимы. Тем не менее для

задачи TSP были как разработаны специальные операторы скрещивания, так и адаптированы некоторые стандартные операторы.

В рамках работы реализованы и экспериментально сравниваются несколько классических операторов скрещивания для задачи TSP, описанных в литературе [1, 2].

2.5.1 Стандартный однородный кроссовер, адаптированный для перестановок

Имея две последовательности одинаковой длины проще всего скомбинировать их случайно. На каждой позиции i с вероятностью $1/2$ выбрать из какого родителя взять элемент. Но в случае комбинации перестановок результат может перестать быть перестановкой. Поэтому предлагается выбирать город и помещать в потомка, если он ещё не был использован; в противном случае позиция временно остаётся пустой. После первого прохода все незаполненные позиции заполняются оставшимися городами случайно. Такой метод сохраняет часть относительного порядка городов, но слабо учитывает соседство вершин в туре, поэтому его эффективность для TSP ограничена.

2.5.2 Двухточечный кроссовер (Order Crossover, OX)

Более естественным для задачи коммивояжёра считается оператор OX (Order Crossover), предложенный специально для перестановок [2]. Процедура выполняется следующим образом:

1. Выбираются две случайные точки разреза i и j ($1 \leq i < j \leq n$).
2. Потомок наследует фрагмент тура между этими точками непосредственно от первого родителя (города на позициях $i, \dots, j$ копируются без изменений).
3. Начиная с позиции $j + 1$, оставшиеся позиции тура заполняются городами второго родителя, взятыми в порядке их следования во втором родителе, начиная с той же позиции $j + 1$, но пропуская те города, которые уже присутствуют в потомке. При достижении конца перестановки просмотр продолжается циклически с начала.

Данный оператор сохраняет не абсолютные позиции городов, а их относительный порядок (циклическую последовательность) и фрагменты маршрута, что хорошо согласуется с природой TSP.

2.5.3 Усредняющий кроссовер (Particle-Like Crossover)

Операторы, описанные выше, стремятся сохранить часть рёберной структуры родителей (OX) или пытаются собрать допустимую перестановку из случайной маски (Uniform). В качестве альтернативы был реализован оператор, вдохновлённый методами роя частиц и кодированием решений с помощью случайных n -мерных векторов. Мы называем его **Particle-Like Crossover**.

Идея состоит в том, чтобы представить каждое родительское решение не прямым порядком городов, а набором вещественных чисел — которые задают позиции каждого города на числовой оси. Для каждого родителя генерируется случайный вектор из n различных чисел. Городам родительского тура присваиваются эти числа в соответствии с порядком их следования: первый город получает наименьшее число, последний — наибольшее.

После этого для каждого города вычисляется среднее арифметическое его рангов в двух родителях. Итоговый порядок городов в потомке определяется простой сортировкой городов по этому среднему значению. Таким образом, город окажется тем левее в маршруте, чем в среднем левее он располагался у родителей. Оператор порождает двух потомков, повторяя процедуру дважды с разными случайными векторами рангов.

Данный метод не сохраняет рёбра в явном виде, но зато наследует глобальный порядок городов. Такой кроссовер сглаживает разногласия между родителями, подобно тому как в оптимизации роем частиц точка смещается в сторону средневзвешенного положения лучших решений.

2.6 Оператор мутации

Мутация вносит небольшие случайные изменения в решения-потомки, выполняя две основные функции: во-первых, она может случайно улучшить особь, действуя подобно локальному поиску малой глубины; во-вторых, она препятствует преждевременной утрате разнообразия в популяции, без которой алгоритм может сойтись к посредственному локальному оптимуму, далёкому от глобального [1].

В контексте задачи коммивояжёра оператор мутации заключается в локальной перестройке тура. Мы рассматриваем и сравниваем в экспериментальной части следующие варианты мутации, представляющие собой один или несколько локальных ходов.

2.6.1 Обмен двух городов (Swap)

Простейший оператор: случайно выбираются два различных индекса i и j в перестановке, и соответствующие города меняются местами. Это соответствует одной транспозиции и является классическим примером мутации для задач на перестановках [2].

2.6.2 Инверсия участка маршрута (Inversion / 2-opt)

Выбираются два индекса $i < j$, и порядок городов в подпоследовательности от i до j меняется на противоположный. Эта операция эквивалентна одному шагу локального поиска **2-opt**, а потому может приводить к существенному улучшению тура, если удалены два пересекающихся ребра и восстановлена допустимая структура цикла. В меметических алгоритмах подобная мутация часто заменяется систематическим применением **2-opt** или **3-opt** ко всем потомкам [1].

2.6.3 Перемешивание подпоследовательности (Scramble)

Случайно выбранный непрерывный фрагмент тура подвергается случайной перестановке городов внутри него. Этот оператор вносит большее разнообразие, чем одиночный обмен или инверсия, и помогает преодолевать локальные барьеры, когда популяция находится на плато.

2.6.4 Параметры мутации

Интенсивность мутации обычно регулируется вероятностью применения оператора к потомку или долей генов, подвергаемых изменению. Следуя [1], мы задаём *частоту мутации* p_m , которая определяет среднюю долю позиций, затрагиваемых мутацией. При $p_m = 5\%$ это означает, что примерно $p_m \cdot n$ элементов перестановки будут случайным образом переставлены (например, путём выполнения $\lfloor p_m \cdot n/2 \rfloor$ случайных обменов). Более высокое значение p_m замедляет сходимость, но позволяет популяции дольше сохранять генетическое разнообразие и, в конечном счёте, находить более качественные решения.

2.7 Меметические алгоритмы

Меметический алгоритм (МА) расширяет генетический алгоритм, встраивая локальную оптимизацию потомков (например, **2-opt** в TSP) [1]. Обычно применяется *ламарковская модель*: улучшенное решение замещает исходное, что быстро повышает качество популяции. Платой за это явля-

ется риск преждевременной сходимости: интенсивный локальный поиск может свести всю популяцию в один бассейн притяжения. Для сохранения баланса используют частичный ламаркизм (улучшаются не все особи), запрет дубликатов и адаптивное управление глубиной поиска.

В задаче коммивояжёра МА эффективно сочетает глобальное исследование, обеспечиваемое популяционными операторами, с локальной доводкой решений, что позволяет достигать высокого качества даже на крупных экземплярах [1].

3 Реализация алгоритма

Программный комплекс реализован на языке Kotlin с использованием корутин (coroutines) для параллельного выполнения независимых запусков и эволюционных операторов. Основными целями при проектировании были модульность и возможность легко заменять отдельные эвристики — селекцию, скрещивание, мутацию, локальный поиск, не затрагивая общую логику алгоритма.

3.1 Архитектура

Ядро системы составляют несколько ключевых интерфейсов, определённых в пакете `model`:

- **Solution** — маркерный интерфейс для решения; в задаче TSP ему соответствует класс `Tour<V>`, хранящий упорядоченный список вершин.
- **Problem<S>** — абстракция задачи, предоставляющая метод оценки `fitness(s: S): Double`.
- **TSP<V>** — конкретизация **Problem** для задачи коммивояжёра; именно здесь вычисляется длина гамильтонова цикла.

Популяция инкапсулирована в классе `Population<S, P>`, который поддерживает добавление, замену наихудшего решения и полную перезапись особей.

Популяционный менеджер. Интерфейс `PopulationManager<S, P>` отвечает за хранение популяции (или нескольких популяций) и её эволюцию. Реализованы два менеджера:

- `SimplePopulationManager` — одна обычная популяция фиксированного размера;
- `IslandPopulationManager` — островная модель: несколько независимых популяций, обменивающихся лучшими особями с заданной интенсивностью (миграция).

Цикл эволюции. Интерфейс `EvolutionCycle<S, P>` определяет, как именно одно поколение сменяет другое. Две классические схемы реализованы классами:

- `ClassicCycle` — классический ГА с полной заменой популяции и элитизмом;
- `SteadyStateCycle` — монотонный ГА, порождающий ограниченное число потомков и заменяющий худших особей.

Операторы. Все операторы, а именно — селекция (`Selection`), скрещивание (`Crossover`) и постобработка (`PostProcessing`) — реализованы как объекты, удовлетворяющие соответствующим интерфейсам. Постобработка (мутация и локальный поиск) дополнительно поддерживает композицию: операторы можно объединять последовательно (`Combine`) и запускать с заданной вероятностью (`Probably`). Это позволяет строить цепочки вида «мутация, затем с вероятностью p локальный поиск 2-opt», не меняя код самого цикла эволюции.

3.2 Конфигурирование тестовых запусков через JSON

В ходе работы проводится сравнение эффективности алгоритмов в зависимости от выбранных операторов и их настроек. Чтобы упростить описание конфигураций и позволить задавать их, не меняя исходный код, конфигурации описываются в формате JSON. Каждый эксперимент описывается тремя секциями: общие настройки (количество повторов, максимальное число поколений, частота логирования и др.), список алгоритмов и список тестовых задач. Каждый алгоритм задаётся именем, а также спецификациями популяционного менеджера, цикла эволюции, операторов селекции, кроссовера, мутации и локального поиска в виде компактных строковых идентификаторов (например, `"OX"`, `"Swap"`, `"HillClimbing2Opt"`). При необходимости можно переопределить любой общий параметр (число поколений, частоту мутации, вероятность локального поиска и т.п.) индивидуально для алгоритма. Такой подход

позволяет описывать десятки комбинаций эвристик без изменения исходного кода.

Файл конфигурации парсится в `Main.kt` с помощью библиотеки `kotlinx.serialization`. Строковые имена преобразуются в конкретные реализации интерфейсов (например, `"island"` → `IslandPopulationManager`, `"ParameterizedTournament"` → `ParameterizedTournamentSelector` с заданным давлением S). Объект `AlgorithmConfig`, получаемый после парсинга, содержит фабрики популяционного менеджера и эволюционного цикла, что гарантирует полную независимость параллельных запусков.

3.3 Сбор метрик и визуализация

Для анализа эффективности эвристик, в процессе каждого запуска `MetricsCollector` с заданной частотой (по умолчанию каждые 10 поколений) записывает в CSV-файл строку вида `generation, best, average, unique_edges`. Показатель `unique_edges` — это усреднённая по парам особей доля уникальных ориентированных рёбер, вычисляемая в `DiversityUtils`. Он служит мерой генетического разнообразия популяции в пространстве рёберных признаков и позволяет отслеживать риск преждевременной стагнации.

По завершении запусков результаты обрабатываются скриптом, написанным на языке Python 3 — `plot_results.py`. Он загружает CSV-файлы всех запусков одного алгоритма, приводит их к общему множеству поколений (если разные повторы имели разную длину) и строит графики:

- сходимость лучшего значения целевой функции (линейная и логарифмическая шкалы);
- сходимость в увеличенном хвосте (zoom tail), где все алгоритмы близки к своему минимуму;
- динамика рёберного разнообразия (оригинальный масштаб и масштабированный под размах значений).

На каждом графике полупрозрачными линиями показаны индивидуальные запуски, жирной чёрной — среднее по запускам, пунктиром — медиана. Для сравнения нескольких алгоритмов строятся аналогичные графики, на которых каждая конфигурация отображается своим цветом. Все изображения сохраняются в директорию `plots/` с иерархией по датасетам, алгоритмам и именованным группам сравнения.

Для автоматизации рутинного попарного анализа реализован скрипт `testing_comparisons.py`. В нём заданы группы сравнений, соответствующие группам, отличающимся только одним оператором или параметром - с целью выявить, как этот оператор или параметр влияет на работу алгоритма. Для заданного датасета скрипт последовательно вызывает `plot_results.py` с соответствующими именами алгоритмов, порождая полный набор сравнительных графиков. Такая система позволила систематически изучить влияние каждого компонента алгоритма на скорость сходимости и итоговое качество решений.

4 Экспериментальное исследование

Первая серия экспериментов была проведена на наборе данных `tsp_200_2`, содержащем 200 городов. Для каждого сочетания операторов выполнялось три независимых запуска по 400 поколений. Популяция везде, где не оговорено обратное, состояла из 24 особей, инициализированных случайными перестановками. Частота мутации, если не указано иное, равнялась 0.1. В качестве локального поиска использовался алгоритм `2-opt`, запускаемый до достижения локального минимума (жадный спуск). Вероятность применения локального поиска к потомку обозначена далее как p_{LS} .

Сравнение запусков проводилось попарно: в каждой группе менялся ровно один компонент алгоритма, а все прочие оставались фиксированными. Это позволило выделить влияние конкретного оператора или параметра, не смешивая его с эффектами от других частей алгоритма.

Для каждого запуска с шагом в 10 поколений записывались три показателя: длина лучшего найденного тура, средняя длина тура по популяции и рёберное разнообразие — доля уникальных ориентированных рёбер, усреднённая по всем парам особей. Разнообразие служит индикатором того, насколько популяция ещё способна исследовать новые участки пространства решений; падение разнообразия до нуля обычно означает, что все особи стали одинаковыми и дальнейший прогресс возможен только за счёт мутаций.

4.1 Вероятность локального поиска

Для начала было проверено, как на результат влияет частота применения локального поиска (`2-opt`). Сравнивались семь значений p_{LS} : 0 (чистый GA без локального поиска), 0.01, 0.05, 0.1, 0.2, 0.5 и 1.0. Остальные операторы были зафиксированы: OX-кроссовер, Swap-мутация, несме-

щённый турнирный отбор, классический цикл с полной заменой поколения.

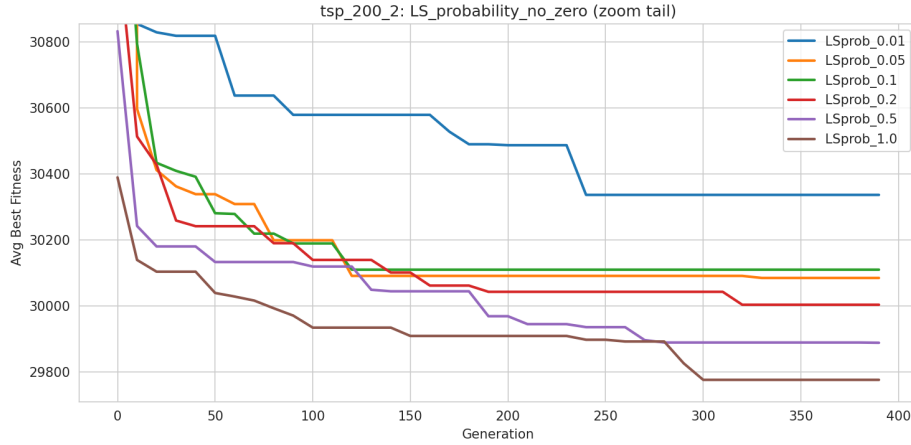


Рис. 2: Сходимость лучшего решения при разной вероятности локального поиска.

Вариант с $p_{LS} = 0$, то есть без какого-либо локального поиска, показал себя несравнимо хуже всех остальных: его лучший тур имеет длину 250 тысяч, в то время как даже при $p_{LS} = 0.01$ длина кратчайшего тура опускается до значений порядка 30000. Дальнейшее увеличение p_{LS} монотонно повышало и итоговое качество, и скорость сходимости. Наилучший результат был получен при $p_{LS} = 1.0$, то есть когда локальный поиск применялся к каждому потомку.

Платой за высокое качество стало снижение разнообразия (рис. 3). Чем чаще включался 2-opt, тем быстрее популяция теряла уникальные рёбра и стягивалась к небольшому числу локальных оптимумов. Любопытно, что вне зависимости от вероятности локального поиска разнообразие не уходило в ноль, а стабилизировалось на некотором ненулевом уровне. Это говорит о том, что мутации и кроссовер сами по себе поддерживают определённый фон «генетического шума», не позволяя популяции полностью выродиться.

4.2 Сравнение операторов мутации

Для сравнения операторов мутации — Swap, Inversion и Scramble — были проведены запуски с $p_{LS} = 0$ (чистый GA) и $p_{LS} = 1.0$ (меметический алгоритм с максимальным локальным поиском).

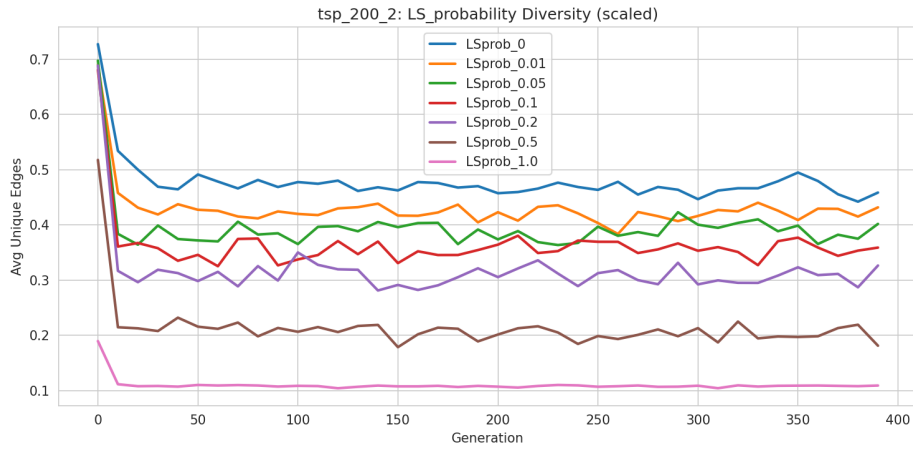


Рис. 3: Динамика рёберного разнообразия при разной вероятности локального поиска.

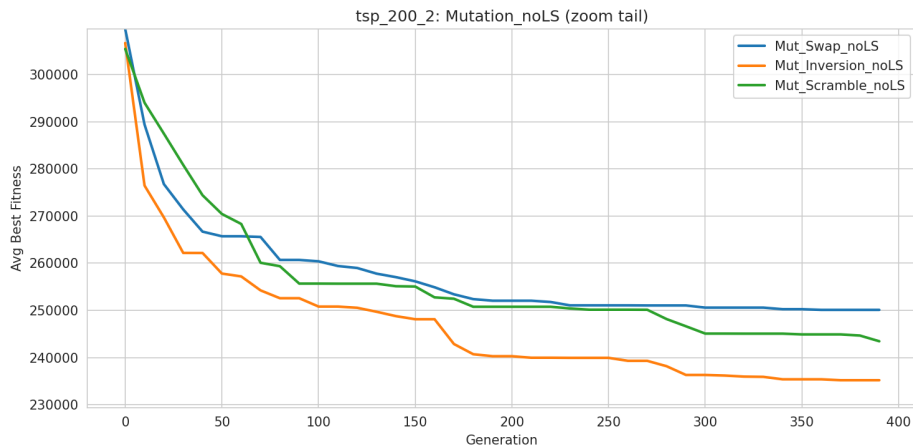


Рис. 4: Сравнение мутаций без локального поиска.

В отсутствие локального поиска (рис. 4) наилучший результат показала инверсия. Это предсказуемый результат: переворот участка маршрута эквивалентен одному шагу **2-opt**, и в условиях, когда систематического улучшения не происходит, такая мутация частично берёт на себя роль локального оптимизатора. Swap и Scramble без локального поиска отработали заметно хуже.

Ситуация изменилась, когда локальный поиск стал применяться к каждому потомку (рис. 5). Здесь вперёд вышел Scramble, несколько опередив и Swap, и Inversion. Объяснение, по-видимому, таково: когда каждый потомок всё равно проходит через **2-opt**, мутация перестаёт быть



Рис. 5: Сравнение мутаций с локальным поиском.

ответственной за точную доводку решения. Вместо этого её главной задачей становится выбивание популяции из локальных оптимумов — а с этим Scramble справляется лучше, поскольку вносит более серьёзные возмущения в структуру тура. Swap и Inversion, будучи более консервативными, чаще оставляют потомка в том же области локального решения, откуда 2-opt возвращает его обратно в тот же локальный минимум.

4.3 Островная модель против простой популяции

В рамках экспериментов было произведено сравнение простой модели популяции из 24 особей с островной моделью, состоящей из четырёх островов по 6 особей каждый. Сравнение проводилось как без локального поиска, так и с $p_{LS} = 1.0$.

Без локального поиска (рис. 6) островная модель показала значительно лучшую сходимость. Это согласуется с известным фактом: изоляция субпопуляций замедляет распространение одного доминирующего решения и позволяет разным островам исследовать разные участки пространства. Интересно, что рёберное разнообразие на островах оказалось ниже, чем в простой популяции, хотя интуитивно ожидалось обратное. Вероятно, на маленьких островах (всего по 6 особей) вырождение наступает быстрее, но миграция всё равно успевает перенести лучшие находки между островами до полной стагнации.

При включённом локальном поиске (рис. 7) островная модель тоже оказалась лучше, хотя разрыв сократился. Мы предполагаем, что с ростом размера задачи и увеличением числа поколений преимущество ост-

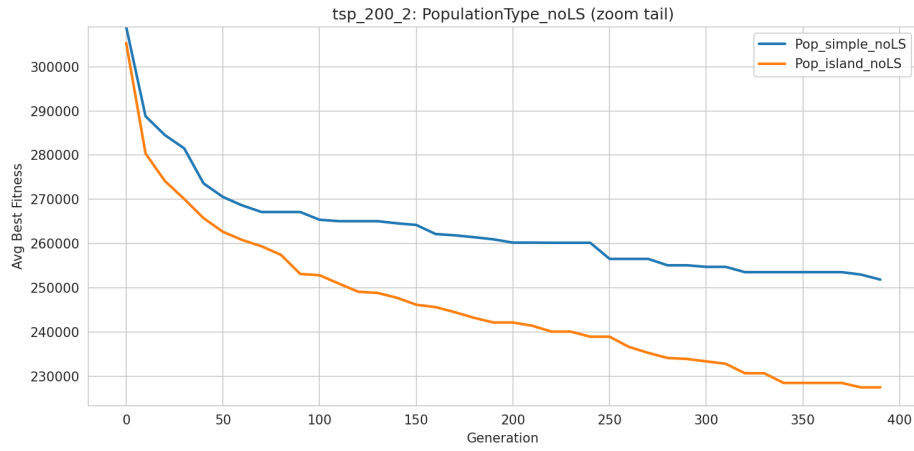


Рис. 6: Простая и островная популяции без локального поиска.

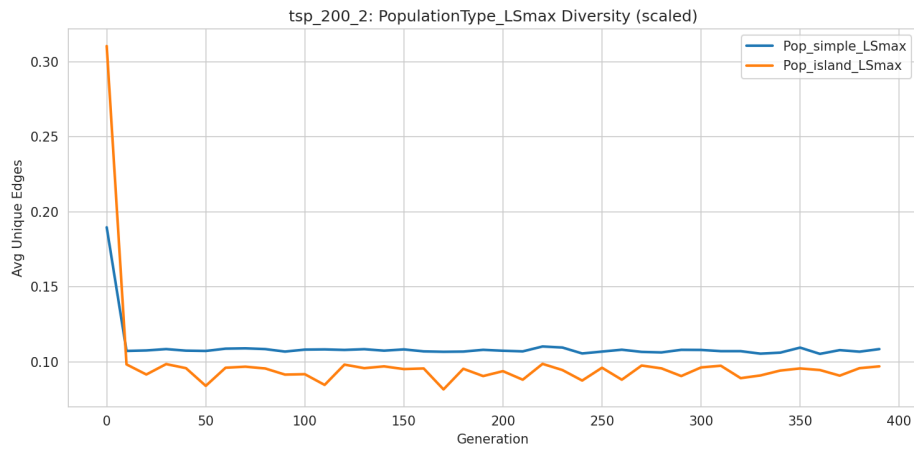


Рис. 7: Простая и островная популяции с локальным поиском.

ровов может стать более выраженным, так как на крупных задачах разнообразие исчерпывается медленнее и у островной модели больше шансов найти несколько принципиально различающихся хороших решений.

4.4 Влияние размера популяции

Мы протестировали три размера популяции — 12, 24 и 48 особей — снова в двух режимах: без локального поиска и с $p_{LS} = 1.0$.

Результат на первый взгляд противоречит интуиции: чем меньше популяция, тем лучше итоговое качество при фиксированном числе поколений (рис. 8). Ту же картину мы наблюдали и при $p_{LS} = 1.0$ (рис. 9).

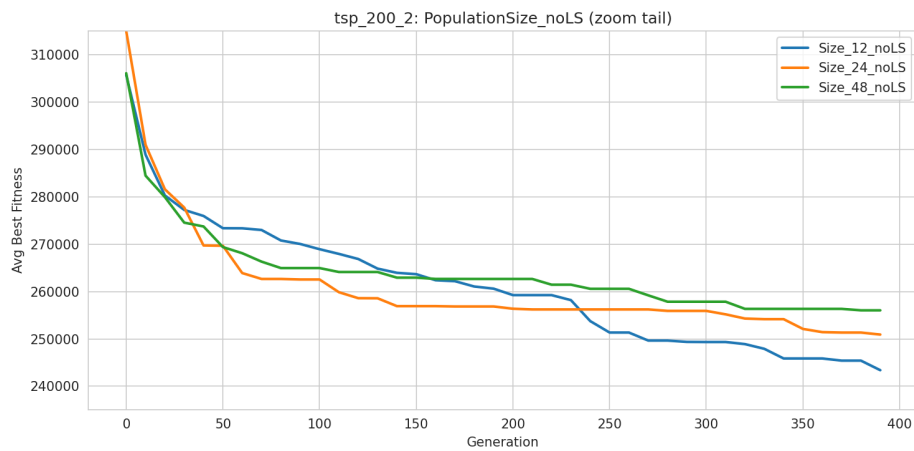


Рис. 8: Влияние размера популяции без локального поиска.

Это можно объяснить тем, что на сравнительно простых задачах при больших популяциях влияние скрещивания и отбора сильнее, чем разнообразие, которое вносит мутация - гены локального минимума слишком быстро распространяются. Именно эту проблему решает островная популяция.



Рис. 9: Влияние размера популяции с локальным поиском.

Практический вывод таков: если время ограничено и число поколений фиксировано, выгоднее держать популяцию умеренного размера. Если же важно сохранить способность выходить из плато на длинных дистанциях, имеет смысл увеличить популяцию, смирившись с более медленной сходимостью в начале.

4.5 Классический цикл против монотонного

Две схемы обновления поколений — классическая (полная замена популяции каждое поколение) и монотонная (только 10 потомков на поколение, замещающих худших особей) — тестировались в тех же двух режимах.

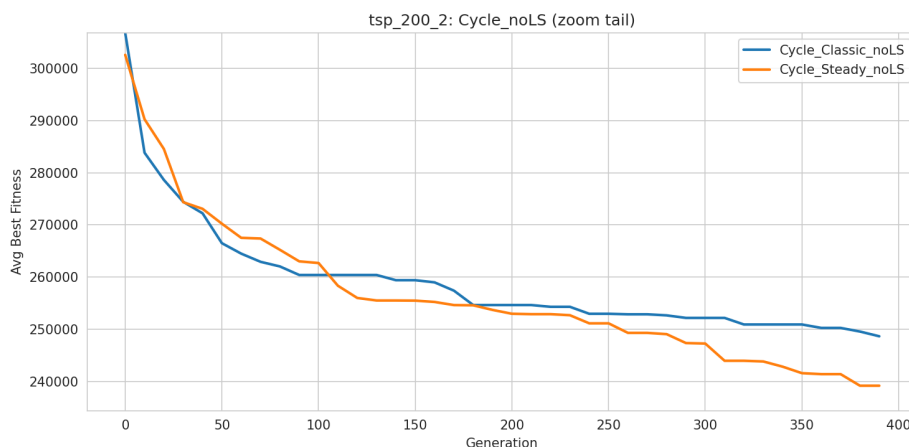


Рис. 10: Классический и монотонный циклы без локального поиска.

Без локального поиска (рис. 10) монотонный цикл заметно опередил классический. Это можно объяснить тем, что такой алгоритм, немедленно внедряя удачные решения в популяцию, обеспечивает более высокое селективное давление и не тратит время на хранение заведомо плохих особей. Разнообразие при этом ожидаемо ниже — популяция быстрее «забывает» проигравших.

С локальным поиском (рис. 11) ситуация становится обратной: классический цикл не только догнал, но и перегнал монотонный. Похоже, что когда каждый потомок подвергается глубокой оптимизации, монотонный цикл становится слишком «жадным». Он немедленно закрепляет первое же хорошее решение, и популяция, состоящая из небольшого числа свежих потомков и множества старых особей, быстро теряет способность к исследованию. Классический же цикл, обновляя сразу всю популяцию, даёт больше шансов появиться структурно различным решениям, и к концу 400 поколений это преимущество перевешивает.

4.6 Сравнение операторов кроссовера

Три кроссовера — OX (Order Crossover), Uniform (однородный, адаптированный для перестановок) и Particle-Like (усредняющий) — сравнивались

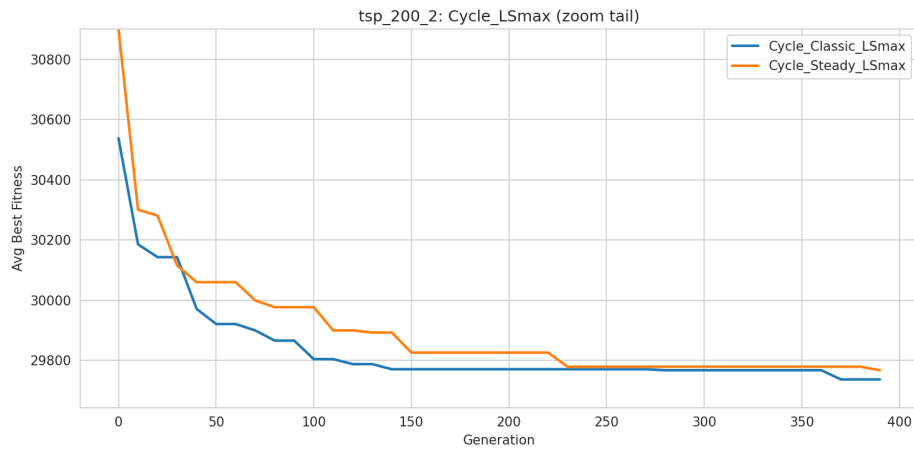


Рис. 11: Классический и монотонный циклы с локальным поиском.

между собой.



Рис. 12: Сравнение кроссоверов без локального поиска.

Без локального поиска (рис. 12) ОХ оказался явным лидером. Uniform и особенно Particle-Like кроссовер заметно отставали. Particle-Like при этом демонстрировал самое высокое разнообразие, что говорит о его неспособности эффективно комбинировать хорошие фрагменты родительских туров — потомки получались слишком «усреднёнными» и теряли ценные рёбра.

При $p_{LS} = 1.0$ разница между кроссоверами сократилась (рис. 13), потому что 2-opt в значительной степени исправлял недостатки потомков, полученных любым способом. Тем не менее ОХ сохранил небольшое



Рис. 13: Сравнение кроссоверов с локальным поиском.

преимущество над Uniform, а Particle-Like остался худшим. Рёберное разнообразие у всех трёх кроссоверов при интенсивном локальном поиске было практически одинаковым и очень низким.

4.7 Операторы селекции

Мы сравнили четыре варианта турнирной селекции: простой бинарный турнир (Tournament), несмещённый бинарный турнир (Unbiased) и параметризованный турнир с давлением $S = 0.75$ (Par75) и $S = 0.90$ (Par90). В отсутствие локального поиска результаты всех четырёх были довольно близки. Наилучшее качество показал несмещённый турнир, наихудшее — Par75. Последний, впрочем, сохранял самое высокое разнообразие, что логично: меньшее давление отбора означает, что выживает больше разных решений.

При $p_{LS} = 1.0$ лучшим оказался простой бинарный турнир. Разнообразие у всех селекторов было практически одинаковым и очень низким — локальный поиск нивелировал различия, быстро подтягивая любые отобранные решения к локальным оптимумам.

4.8 Инициализация популяции

Два способа начальной генерации — полностью случайные перестановки и жадный алгоритм ближайшего соседа — дали предсказуемый, но от этого не менее важный результат. Без локального поиска жадная инициализация дала колоссальный выигрыш: начальное лучшее решение было

на порядок короче, чем при случайной инициализации, и это преимущество сохранялось на протяжении всех 400 поколений. Интересно, что рёберное разнообразие в обоих случаях сошлось к одному и тому же значению около 0.45, только при жадной инициализации — снизу, а при случайной — сверху.

При $p_{LS} = 1.0$ жадная инициализация неожиданно оказалась несколько хуже случайной. Вероятно, дело в том, что жадный алгоритм загоняет стартовую популяцию в глубокий, но не глобальный локальный минимум, из которого даже **2-opt** не может выбраться без помощи популяционных операторов. Случайная же инициализация даёт более разнообразные начальные точки, и локальный поиск в сочетании с кроссовером находит более удачные комбинации.

4.9 Промежуточные выводы

Подведём итог того, что мы увидели на 200 городах:

1. Локальный поиск — самый влиятельный компонент. Даже редкое применение **2-opt** ($p_{LS} = 0.01$) даёт скачок качества. Чем чаще локальный поиск, тем лучше итоговый результат, но тем быстрее падает разнообразие.
2. Без локального поиска лучшая мутация — **Inversion**, с локальным поиском — **Scramble**. Это отражает смену роли мутации с «доводчика» на «возмутителя».
3. Островная модель стабильно лучше простой популяции, хотя её преимущество сокращается при интенсивном локальном поиске.
4. При фиксированном числе поколений меньшая популяция сходится быстрее.
5. Монотонный цикл эффективнее классического без локального поиска, но уступает ему при $p_{LS} = 1.0$.
6. Кроссовер **OX** — наилучший выбор для TSP среди рассмотренных. **Uniform** и **Particle-Like** заметно хуже, особенно без локального поиска.
7. Турнирные селекции показали близкие результаты; при интенсивном локальном поиске различия практически стираются.
8. Жадная инициализация критически важна, если локальный поиск не используется, но может навредить при $p_{LS} = 1.0$.

Эти выводы были получены на задаче умеренного размера с небольшим числом повторов, поэтому к ним не стоит относиться как к строгим статистическим утверждениям. Скорее, это наблюдаемые тенденции, которые мы затем проверили на более крупном датасете.

4.10 Масштабирование на 574 города

Для проверки насколько выводы, полученные на 200 городах, переносятся на более крупные задачи, была проведена вторая серия экспериментов на наборе `tsp_574_1` (574 города). Вычислительная сложность одного поколения на 574 городах примерно в восемь раз выше, чем на 200, поэтому полное повторение всех тестов первой серии потребовало бы значительно больше времени. Мы ограничились наиболее важными сравнениями, а для ускорения запусков число поколений для большинства конфигураций с локальным поиском было сокращено до 80. Однако предварительные результаты показали, что островная и простая популяции при 80 поколениях ещё не успевают разойтись, поэтому для их сравнения число поколений было увеличено до 200 — именно эти данные и обсуждаются ниже.

На рисунке 14 сравниваются три режима локального поиска: чистый GA ($p_{LS} = 0$), $p_{LS} = 0.1$ и $p_{LS} = 1.0$. Результат подтвердил решающую роль локальной оптимизации. Без неё алгоритм практически не улучшал начальное решение и к 400-му поколению длина тура оставалась на порядок выше, чем при любом $p_{LS} > 0$. Вариант $p_{LS} = 0.1$ заметно улучшал маршрут, однако за 400 поколений так и не догнал $p_{LS} = 1.0$, который уже к 80-му поколению находил значительно более короткий тур. Таким образом, на большой размерности полный локальный поиск каждого потомка окупается даже при сравнительно небольшом числе поколений.

Сравнение мутаций при $p_{LS} = 1.0$ (Swap, Inversion, Scramble) дало ту же картину, что и на 200 городах: Scramble стабильно опережал остальных (рис. 15). Преимущество Scramble проявлялось как в скорости сходимости, так и в итоговом качестве. Это подтверждает гипотезу о том, что при интенсивном локальном поиске мутация должна в первую очередь разрушать локально оптимальные структуры, а не пытаться их улучшить.

Схемы обновления поколений — классическая и монотонная — тестировались как без локального поиска, так и с $p_{LS} = 1.0$ (рис. 16). Без локального поиска монотонный цикл ожидаемо опережал классический. При $p_{LS} = 1.0$ классический цикл показал несколько лучший результат. Похоже, что при ограниченном числе поколений более «жад-

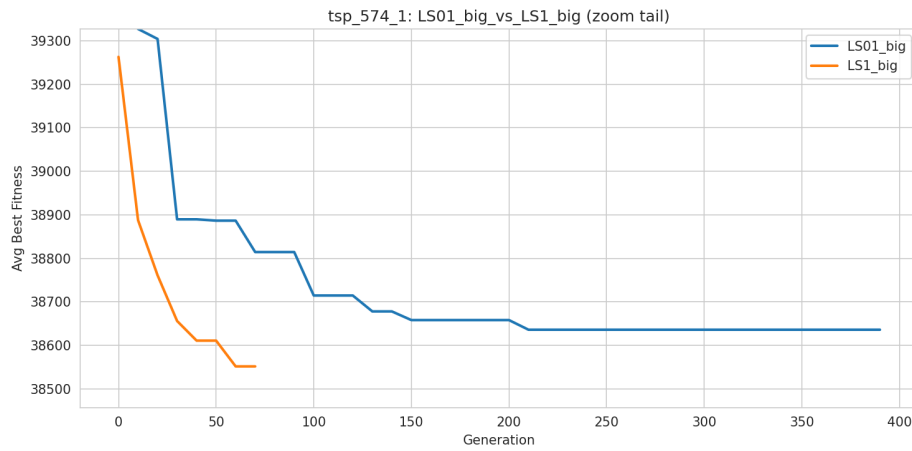


Рис. 14: Сравнение вероятностей локального поиска на 574 городах.

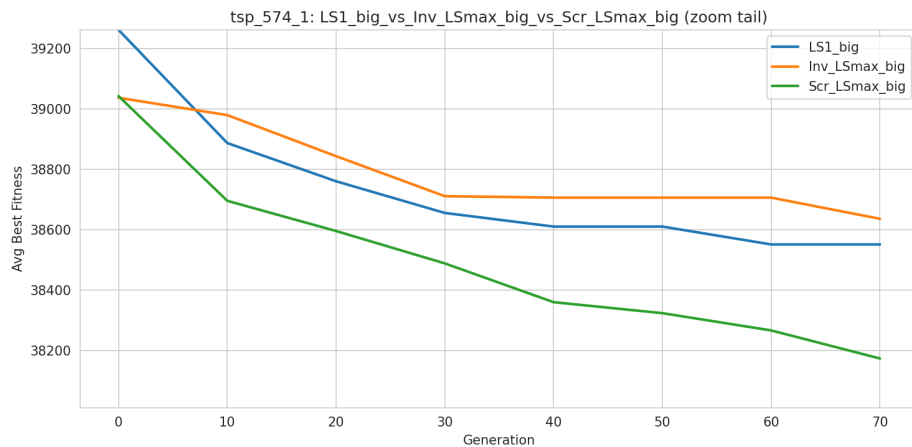


Рис. 15: Сравнение мутаций с локальным поиском на 574 городах.

ный» монотонный цикл успевает реализовать своё преимущество ранней сходимости, тогда как на более длинной дистанции классический цикл выигрывает за счёт лучшего исследования.

Островная и простая популяции при $p_{LS} = 1.0$ сравнивались на протяжении 200 поколений (рис. 17). В первой половине запуска простая популяция шла впереди, быстрее находя и распространяя удачные решения. Однако ближе к середине её прогресс замедлился, и инициатива перешла к островной модели. К 200-му поколению островная популяция значительно опередила простую, причём её кривая сходимости сохраняла заметный наклон, тогда как простая популяция практически вышла

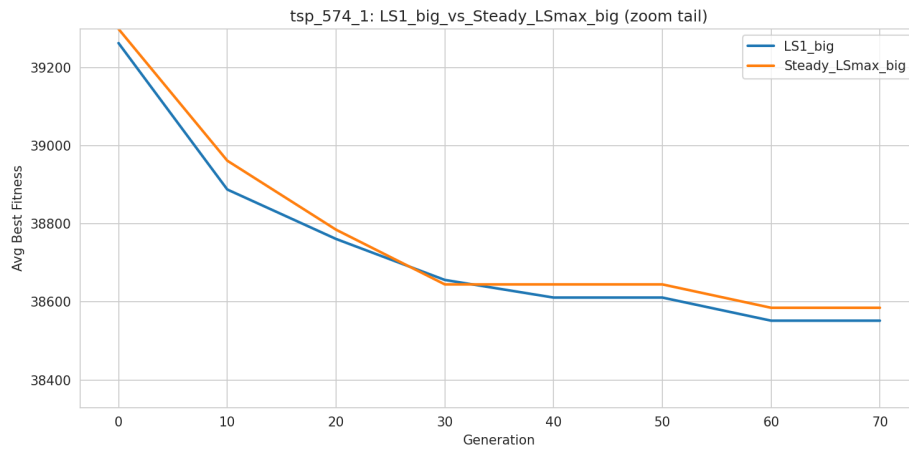


Рис. 16: Классический и монотонный циклы на 574 городах.

на плато. Такое поведение согласуется с теоретическими ожиданиями: на ранних этапах простая модель популяции быстрее распространяет хорошие гены, но затем вырождается и застревает в локальном оптимуме; островная же модель, благодаря изоляции субпопуляций, дольше сохраняет разнообразие и в итоге находит лучшее решение.

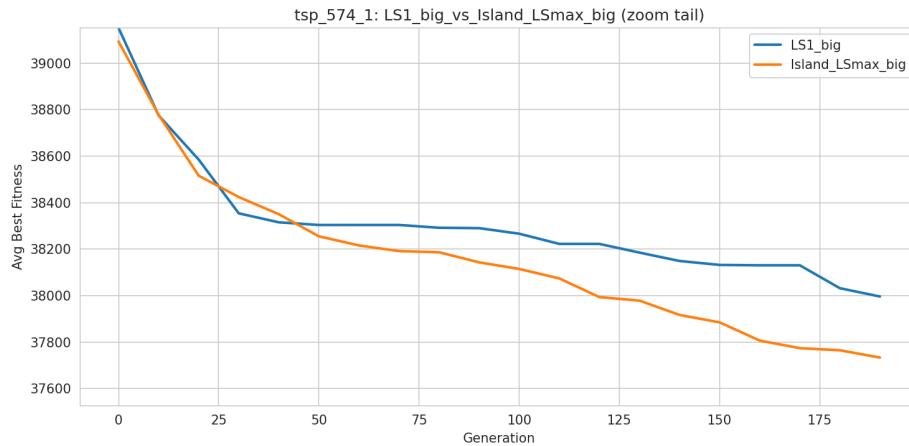


Рис. 17: Островная и простая популяции при $p_{LS} = 1.0$ на 574 городах.

Наконец, мы проверили, насколько большой размер популяции способен заменить островную модель в качестве средства борьбы с вырождением. Для этого была запущена конфигурация со 128 особями, Scramble-мутацией и $p_{LS} = 1.0$ (рис. 18). За 100 поколений большая популяция успела найти более короткий тур, чем стандартная (24 особи) за те же

100 поколений, что подтверждает тезис о лучшем сохранении разнообразия в многочисленной популяции. Однако за 200 поколений стандартная популяция сумела продвинуться дальше и в итоге показала лучший результат, причём затратила на это значительно меньше времени. Большая популяция — это дорогое удовольствие: каждое поколение требует локального поиска на 128 особях, и даже за 100 поколений такое вычисление длится дольше, чем 200 поколений с 24 особями. Таким образом, островная модель оказывается более экономичным способом поддержания разнообразия, нежели простое раздувание популяции.

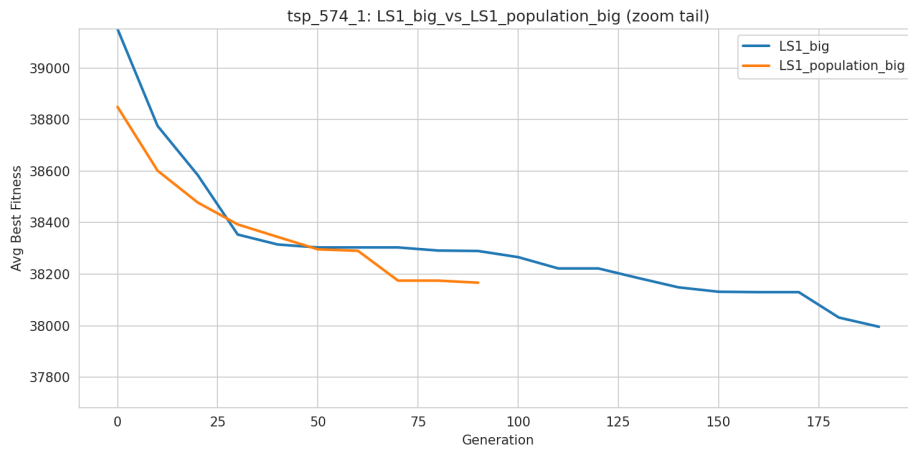


Рис. 18: Стандартная популяция против большой.

4.11 Общие выводы

Суммируя наблюдения на двух датасетах, можно сформулировать следующие основные результаты работы.

1. Локальный поиск является ключевым компонентом меметического алгоритма для задачи коммивояжёра. Даже однократное применение **2-opt** к небольшой доле потомков радикально улучшает качество решения. При фиксированном бюджете поколений наилучший результат достигается при $p_{LS} = 1.0$.
2. При отсутствии локального поиска лучшей мутацией оказывается Inversion, фактически выполняющая шаг **2-opt**. При интенсивном локальном поиске лидерство переходит к Scramble, который вносит достаточно сильные возмущения, чтобы выводить популяцию из локальных оптимумов.

3. Островная модель популяции превосходит простую на длинных дистанциях. На ранних этапах простая популяция может идти впереди, но со временем вырождается, тогда как островная модель дольше сохраняет способность к улучшению и в итоге находит лучшее решение. Особенно заметным это преимущество становится на задачах большого размера.
4. Кроссовер ОХ является предпочтительным выбором для задачи коммивояжёра. Однородный кроссовер (Uniform) и усредняющий (Particle-Like) заметно уступают, особенно в отсутствие локального поиска.
5. Выбор между классической и монотонной схемами зависит от интенсивности локального поиска. Без локального поиска монотонный цикл эффективнее; при $p_{LS} = 1.0$ классический цикл показывает лучшие результаты на длинных дистанциях.
6. Инициализация популяции жадным алгоритмом даёт значительное преимущество, если локальный поиск не используется, но при $p_{LS} = 1.0$ может приводить к преждевременной стагнации в глубоком локальном оптимуме.
7. Популяция меньшего размера быстрее сходится при фиксированном числе поколений, однако большая популяция лучше сохраняет разнообразие и может оказаться предпочтительнее при длительном поиске.

Полученные результаты позволяют рекомендовать для практического решения задачи коммивояжёра меметический алгоритм с ОХ-кроссовером, Scramble-мутацией, островной моделью популяции (при достаточном бюджете времени) и полным локальным поиском ($p_{LS} = 1.0$). При жёстких временных ограничениях можно уменьшить размер популяции и использовать монотонный цикл с умеренной частотой локального поиска. Также стоит отметить, что в нашей реализации из-за параллельности вычисления локального поиска на всех особях в популяции, стоит учитывать количество ядер процессора для подбора оптимального размера популяции и вероятности локального поиска.

5 Заключение

В данной работе подробно исследуется применимость генетических и меметических алгоритмов для решения задачи коммивояжёра (TSP). В хо-

де работы была спроектирована и реализована гибкая модульная архитектура на языке Kotlin, которая позволяет легко комбинировать различные эволюционные операторы, а также была разработана удобная система запуска экспериментов и автоматического сбора метрик.

В ходе масштабного тестирования было подтверждено, что классический генетический алгоритм без локальной оптимизации демонстрирует крайне низкую эффективность на сложных комбинаторных задачах. Внедрение локального поиска (алгоритма **2-opt**) превращает его в мощный меметический алгоритм, способный находить качественные решения за разумное время. При этом ключевым фактором успеха становится соблюдение баланса между интенсивностью поиска (селективным давлением) и сохранением рёберного разнообразия популяции.

Эксперименты привели к нескольким неочевидным и важным выводам:

1. Роль мутации принципиально меняется в зависимости от режима: в чистом **GA** она выступает в роли «доводчика» (где лучше всего работает инверсия), а в меметическом алгоритме — в роли «возмутителя спокойствия» (где лидирует Scramble), выбивающего решения из глубоких локальных оптимумов.
2. Жадная инициализация популяции полезна только при отсутствии локального поиска; при полной меметической оптимизации она провоцирует преждевременную стагнацию, в то время как случайный старт обеспечивает необходимое стартовое разнообразие.
3. Островная модель популяции стабильно превосходит простую за счёт изоляции субпопуляций, что замедляет вырождение рёберной структуры и помогает находить принципиально разные сильные решения.

В качестве направлений для дальнейшего развития проекта можно выделить интеграцию более эффективных процедур локального поиска (как самой тяжёлой вычислительно части), внедрение адаптивного управления параметрами (динамическое изменение вероятности мутации и p_{LS} на основе текущего рёберного разнообразия **unique_edges**), а также проведение экспериментов на более крупных задачах для проверки масштабируемости предложенных подходов.

Список литературы

- [1] Michel Gendreau and Jean-Yves Potvin, editors. *Handbook of Metaheuristics*, volume 2. Springer, 2010.
- [2] Éric D. Taillard. *Design of Heuristic Algorithms for Hard Optimization: With Python Codes for the Travelling Salesman Problem*. Springer Nature, 2023.